

An Implementation of a Pseudoperipheral Node Finder

ALAN GEORGE and JOSEPH W. H. LIU

University of Waterloo

The need to find nodes with large eccentricity arises in many heuristic algorithms for ordering sparse matrix equations. A computer program is presented here for finding such nodes, based on an algorithm due to Gibbs, Poole, and Stockmeyer. Several changes have been made to their original method, and experiments suggest that these changes improve its reliability and execution time.

Key Words and Phrases: sparse linear equations, peripheral nodes, mathematical software, graph algorithm

CR Categories: 4.0, 5.14

1. INTRODUCTION

Many algorithms for finding orderings for sparse symmetric matrices operate on the corresponding undirected graph. These algorithms often require one or more "starting nodes," and for some algorithms experience suggests that nodes which are at maximum or nearly maximum distance apart are good candidates [4, 6-11]. In a recent paper, Gibbs et al. [8] provide a novel heuristic algorithm for finding such nodes. Our objective in this paper is to provide a well-structured flexible implementation of this algorithm which includes some modifications that appear to improve its performance. We include some experiments demonstrating the effect of various modifications to the original algorithm.

We now give some formal definitions and a precise statement of the problem. Let $G = (X, E)$ be an *undirected graph* with the set X of nodes and the set E of undirected edges represented as unordered pairs of nodes. A *path of length k* is an ordered set of distinct nodes $(x_0, x_1, \dots, x_k)$ where $\{x_{i-1}, x_i\} \in E$ for $1 \leq i \leq k$. A graph is *connected* if for each pair of distinct nodes there is a path joining them. Throughout this paper, graphs are assumed to be connected unless we state otherwise.

Consider a connected graph G . The *distance* $d(x, y)$ between two nodes x and y in G is defined to be the length of a shortest path connecting them. Following Berge [2], we define the *eccentricity* of a node x to be the quantity

$$l(x) = \max\{d(x, y) \mid y \in X\}.$$

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires a fee and/or specific permission.

This research was supported in part by the Canadian National Research Council under Grant A8111. Author's address: Department of Computer Science, University of Waterloo, Waterloo, Ont., Canada.

© 1979 ACM 0098-3500/79/0900-0284 \$00.75

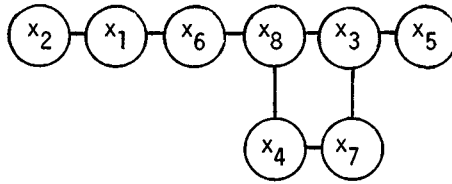


Fig. 1. An 8-node graph

The diameter of G is then defined as

$$\delta(G) = \max\{l(x) \mid x \in X\},$$

or equivalently,

$$\delta(G) = \max\{d(x, y) \mid x, y \in X\}.$$

A node $x \in X$ is said to be a *peripheral node* if its eccentricity is equal to the diameter of the graph, that is, $l(x) = \delta(G)$. Following Gibbs et al. [8], we define a *pseudoperipheral node* to be one whose eccentricity is close to $\delta(G)$. Figure 1 is an example of a graph with diameter five. The nodes x_2 , x_5 , and x_7 are the only peripheral nodes.

With this terminology, our objective may be restated as the implementation of an efficient heuristic algorithm to determine pseudoperipheral nodes.

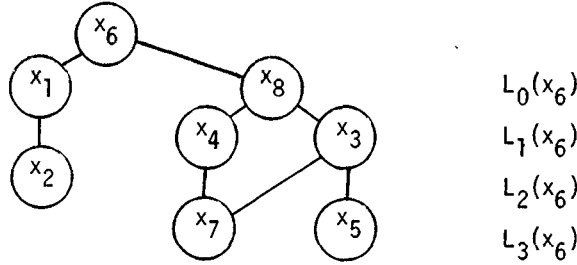
It should be emphasized that both the original algorithm of Gibbs et al. [8] and our modified version are heuristic; neither is *guaranteed* to find a peripheral node, or even one that is close to being peripheral. Nevertheless, the nodes produced usually do have high eccentricity and are effective as starting nodes for many ordering algorithms. In any case, except for some fairly trivial situations, there seems to be no reason to expect that peripheral nodes are any better as starting nodes than the nodes found by our algorithm anyway. Finally, in many situations it is probably too expensive to find peripheral nodes even if it was known to be desirable to use them, since the best known algorithm for finding such nodes has a time complexity bound of $O(|X| \cdot |E|)$, which for most sparse matrix applications would be $O(|X|^2)$ [12]. In some sparse matrix applications this would dominate all other phases of the computation [7].

The development of these subroutines has been motivated by their use in several ordering algorithms developed by the authors [6, 7]. The subroutines are used repeatedly by the ordering algorithms, typically $O(|X|^{1/2})$ or even $O(|X|)$ times; so they must be efficient. They are normally applied to only certain subgraphs of the original matrix graph, which means that it must be convenient to apply the subroutines to such subgraphs.

2. SOME RELATED DEFINITIONS AND NOTATIONS

In this section, we introduce some more definitions and notations that are useful in the description of the algorithm. Let $G = (X, E)$ be a connected graph and consider a subset $Y \subset X$. The *adjacent set* of Y is defined to be the set

$$\text{Adj}(Y) = \{x \in X - Y \mid \{x, y\} \in E \text{ for some } y \in Y\}.$$

Fig. 2 A rooted level structure $\mathcal{L}(x_6)$

When $Y = \{y\}$, we use the notation $\text{Adj}(y)$ instead of $\text{Adj}(\{y\})$. The *degree* of a node y is then the number $|\text{Adj}(y)|$, where $|S|$ denotes the cardinality of the set S .

A graph $G' = (X', E')$ is said to be a *subgraph* of $G = (X, E)$ if $X' \subseteq X$ and $E' \subseteq (E \cap (X' \times X'))$. For a subset $Y \subset X$, the *section graph* of Y in G is the subgraph $G(Y) = (Y, E(Y))$, where

$$E(Y) = \{\{y_1, y_2\} \in E \mid y_1, y_2 \in Y\}.$$

Although the graph G is assumed to be connected, its section subgraphs may be disconnected. When a section graph $G(Y)$ is disconnected, it consists of two or more *connected components*.

A key construct in the algorithm described in Section 3 is the rooted level structure [1]. Consider a node $x \in X$. The *rooted level structure* at x is the partitioning of the node set

$$\mathcal{L}(x) = \{L_0(x), L_1(x), \dots, L_{l(x)}(x)\},$$

where $L_0(x) = \{x\}$, and $L_i(x) = \text{Adj}(\bigcup_{k=0}^{i-1} L_k(x))$, $1 \leq i \leq l(x)$, and $l(x)$, the *eccentricity* of x , is sometimes referred to as the *length* of $\mathcal{L}(x)$. It can be verified readily that

$$\bigcup_{k=0}^{l(x)} L_k(x) = X.$$

The *width* of $\mathcal{L}(x)$ is defined to be $\max\{|L_i(x)| : 0 \leq i \leq l(x)\}$. For convenience, we shall denote this quantity by $w(x)$.

In Figure 2 we have the rooted level structure of the graph of Figure 1, rooted at the node x_6 . Note that $l(x_6) = 3$ and $w(x_6) = 3$.

3. DESCRIPTION OF THE ALGORITHM AND SOME MODIFICATIONS

3.1 The Original Algorithm

In [8], Gibbs et al. presented an algorithm to determine pseudoperipheral nodes. Their algorithm is a finite iterative process based on the following observation:

$$y \in L_{R(x)}(x) \Rightarrow l(x) \leq l(y).$$

In our terminology, their algorithm can be described as follows.

Step 1 (Initialization). Choose a node r of minimum degree.

Table I. Execution Times Showing Effect of Short Circuiting and Shrinking Strategy S2

n	N	Original algorithm	Short circuiting	Minimum degree shrink (S2)
15	225	.63	.43	.066
20	400	1.47	.69	.11
25	625	2.87	1.89	.17
30	900	4.93	2.26	.26
35	1225	7.87	5.17	.33
40	1600	11.92	5.25	.44

Step 2 (Generation of level structure). Construct the rooted level structure at r : $\mathcal{L}(r) = \{L_0(r), L_1(r), \dots, L_{l(r)}(r)\}$.

Step 3 (Sort the last level). Sort the nodes in $L_{l(r)}(r)$ in order of increasing degree.

Step 4 (Test for termination). For $x \in L_{l(r)}(r)$ in order of increasing degree, generate $\mathcal{A}(x) = \{L_0(x), L_1(x), \dots, L_{l(x)}(x)\}$. If $l(x) > l(r)$, set $r \leftarrow x$ and go to step 3.

Step 5 (Exit). The node r is the pseudoperipheral node determined.

3.2 Modification by Short Circuiting

For a given graph $G = (X, E)$ and a node $x \in X$, the amount of work required to generate the rooted level structure $\mathcal{A}(x)$ is at least $O(|E|)$, since each edge has to be inspected at least once. If r is the pseudoperipheral node thus determined, it follows that the algorithm requires at least $|L_{l(r)}(r)| \cdot O(|E|)$ units of work. How then can the algorithm be speeded up while producing reasonably good pseudoperipheral nodes?

The actual implementation of the Gibbs et al. algorithm [3] is made more efficient by introducing a "short circuit" technique by which wide level structures are rejected as soon as they are detected. This idea was motivated by the observation that long level structures are usually narrow; moreover, the use of the algorithm by Gibbs et al. involved finding level structures which were *both* long and narrow, so this modification was very natural.

The short circuit modification can be described by changing step 4 as follows.

Step 4' (Test for termination). For $x \in L_{l(r)}(r)$ in order of increasing degree, generate $\mathcal{L}(x) = \{L_0(x), L_1(x), \dots, L_{l(x)}(x)\}$. If $w(x) \leq w(r)$ and $l(x) > l(r)$, set $r \leftarrow x$ and go to step 3.

This modification, which is accomplished by testing if $|L_i(x)| > w(r)$ as each level $L_i(x)$ is generated, has the effect of reducing the second factor in the expression $|L_{l(r)}(r)| \cdot O(|E|)$, since only part of some of the rooted level structures will be generated. The numerical results reported in Table I illustrate how effective this short circuiting technique can be for some problems.

3.3 Shrinking the Last Level

Unfortunately, even with the incorporation of this short circuiting feature, some of the level structures generated may have last levels containing a substantial number of nodes. We observed that often the level structures rooted at these nodes all had similar or identical lengths and widths, but the short circuiting technique did not detect this until a significant fraction of each level structure had been generated.

This led us to speculate whether it might be just as effective to simply choose one, or a small fraction of the last level nodes to test. The following three strategies for "shrinking" the last level were tested on a set of test problems (described in Section 4).

S1. Select a node of minimum degree from each connected component in $G(L_{l(r)}(r))$.

S2. Select a node of minimum degree from $L_{l(r)}(r)$.

S3. Select an arbitrary node from $L_{l(r)}(r)$.

The three new algorithms we tested can be described as follows.

Step 1 (Initialization). Choose an arbitrary node r in X .

Step 2 (Generation of level structure). Construct the rooted level structure at r : $\mathcal{L}(r) = \{L_0(r), L_1(r), \dots, L_{l(r)}(r)\}$.

Step 3 (Shrinking). Shrink $L_{l(r)}(r)$ according to one of the strategies described above (S1, S2, or S3).

Step 4. For each node x in the (shrunk) last level, generate its rooted level structure $\mathcal{L}(x)$.

If $l(x) > l(r)$, put $r \leftarrow x$ and go to step 3.

Step 5 (Exit). The node x is the pseudoperipheral node determined.

In addition to step 3, the algorithm(s) above have two other important changes compared to the original algorithm of Section 3.1. First, step 1 is changed so that an arbitrary node was chosen, rather than one of minimum degree. This change was motivated by our requirement that the subroutine work efficiently on an arbitrary section subgraph of the original given graph. Since the degrees of the nodes in the subgraph are not known until the subgraph is prescribed, we cannot precompute degrees of the nodes, but must compute them for each subgraph processed. The new step 1 usually results in at most one additional level structure being generated, and avoids the initial calculation of the degrees of all the nodes in the subgraph being processed. Of course some of the strategies require the degrees of some of the nodes, so some of this initial calculation is necessary anyway. However, usually only a very small fraction of the nodes have their degrees calculated; our new step 1 reduced execution time substantially, and made little or no difference to the results obtained.

In addition to changes in step 1 and step 3, the original step 5 has also been changed so that node x , the last node for which a level structure is generated, rather than node r , is returned as the pseudoperipheral node. This change was motivated by our applications, which use the level structure generated by the pseudoperipheral node. If we use the original version of the algorithm, we must either retain the level structure rooted at r , and thus use an extra $2N$ storage locations, or else recompute that level structure. Note that $l(x) = l(r)$, so the length of the level structures returned will be the same for either step 5.

Note that our algorithm includes step 4 rather than step 4'; we discarded the short circuiting strategy on the following basis. First, note that for strategies S2 and S3 the short circuiting technique would only operate once, during the generation of the last level structure, rooted at the node which would ultimately be accepted as the pseudoperipheral node. Moreover, we found that it was usually the case that the last level section graphs had only one component, so shrinking strategies S1 and S2 were usually equivalent. As mentioned above, in our applications we desire not only the pseudoperipheral node, but also the level structure it generates, so in these applications the last level structure is needed

Table II. Average Execution Times, Minimum and Average "Diameters" Found by the Five Implementations Run on the 25 Test Examples

Test problem	N	Original			Short circuiting			Strategy S1			Strategy S2			Strategy S3		
		Diameter			Diameter			Diameter			Diameter			Diameter		
		Time	Min-imum	Average	Time	Min-imum	Average	Time	Min-imum	Average	Time	Min-imum	Average	Time	Min-imum	Average
1	936	.21	49	49.0	.15	49	49.0	.22	49	49.0	.20	49	49.0	.21	39	46.9
2	1009	.25	49	49.0	.36	41	46.2	.26	49	49.0	.26	49	49.0	.25	41	48.2
3	1089	.24	65	65.0	.19	65	65.0	.24	65	65.0	.24	65	65.0	.23	65	65.0
4	1440	.32	64	82.9	.32	61	80.2	.36	64	82.9	.35	64	82.9	.31	64	82.0
5	1180	.36	73	73.0	.27	63	72.0	.32	73	73.0	.25	73	73.0	.25	64	71.2
6	1377	.31	113	113.0	.25	94	109.4	.30	113	113.0	.30	113	113.0	.30	113	113.0
7	1138	.63	61	61.0	.34	55	60.4	.25	61	61.0	.26	61	61.0	.25	61	61.0
8	1141	.71	49	49	.55	44	47.8	.25	49	49.0	.25	49	49.0	.25	49	49.0
9	1349	.31	58	58	.24	53	57.1	.31	58	58.0	.31	58	58.0	.32	52	56.2
10	1174	1.35	54	54.0	.80	37	47.9	1.40	54	54.0	.51	54	54.0	.51	54	54.0
11	737	.15	15	15.0	.15	12	14.7	.16	15	15.0	.14	15	15.0	.14	15	15.0
12	744	.32	14	14.0	.20	10	13.3	.36	14	14.0	.16	13	13.5	.15	13	13.5
13	753	.25	13	13.0	.14	9	11.5	.25	13	13.0	.15	13	13.0	.15	13	13.0
14	750	.35	12	12.8	.12	10	11.4	.35	12	12.8	.16	12	12.5	.16	12	12.5
15	748	.21	14	14.0	.16	9	12.4	.20	14	14.0	.17	14	14.0	.17	14	14.0
16	1250	.48	53	53.0	.33	53	53.0	.48	53	53.0	.20	53	53.0	.20	53	53.0
17	796	.22	32	32.9	.15	25	31.0	.23	32	32.9	.14	32	32.8	.13	32	32.8
18	496	.16	9	9.0	.10	7	8.3	.19	9	9.0	.12	7	8.2	.11	7	8.2
19	599	.75	7	7.0	.43	6	6.8	.28	7	7.0	.16	7	7.0	.14	6	6.8
20	683	.14	13	13.0	.11	10	11.9	.14	13	13.0	.14	13	13.0	.14	13	13.0
21	700	.67	9	9.0	.19	7	7.6	.69	9	9.0	.19	8	8.8	.18	7	8.6
22	784	.30	11	11.0	.24	9	10.5	.33	11	11.0	.15	10	10.5	.15	10	10.5
23	900	.48	8	8.0	.60	7	7.4	.51	7	7.9	.27	7	7.9	.27	6	7.5
24	997	.66	9	9.0	.49	7	8.6	.67	9	9.0	.25	8	8.9	.25	8	8.8
25	976	.26	13	13.0	.12	10	11.2	.27	13	13.0	.19	13	13.0	.20	13	13.0

anyway and the short circuiting saves us nothing. A further motivation for discarding the short circuiting strategy is demonstrated by the results given in Table II; sometimes it ignores too much and yields a level structure having substantially fewer levels than the diameter of the graph.

4. SOME EXPERIMENTS

In this section we present some numerical experiments which illustrate some of the points made in the previous section, and which compare the effectiveness of the various modifications proposed. In all our implementations, including the original algorithm and the original with the short circuiting technique employed, the starting node is *arbitrarily selected*. We did this because, as explained in Section 3, we wanted all the implementations to work well on subgraphs, and it is too inefficient to compute the degrees of all the nodes in the subgraph each time the node finder is applied.

Our first example is one which originally provided the motivation for this paper. Consider an n by n grid. Associated with it is a graph having $N = n^2$ nodes located at the intersections of the grid lines. Nodes are connected if they share a common small grid square. Figure 3 contains an example of a 4 by 4 grid and its associated graph.

In Table I are experimental results for the original algorithm, the algorithm with the short circuiting feature, and the algorithm with shrinking strategy S2, applied to the n by n grid for $n = 15, 20, \dots, 40$. Since the three shrinking strategies all produced essentially the same results for this problem we include results only for strategy S2. In all cases the diameter of the graph was found. The starting node used was $\lfloor N/2 \rfloor$.¹ Execution times reported are given in seconds on an IBM 360/75 computer. The programs are all written in Fortran and were compiled using the optimizing version of the IBM H-level compiler.

Although the numbers illustrate convincingly that the short circuiting strategy can be very effective, and that the shrinking strategy can be even more so, some of their effectiveness is due to the regularity of the problem, which inevitably yields at least one relatively large ($O(\sqrt{N})$) last level.

In order to obtain some information about the expected behavior of the algorithms which incorporate the various strategies, we applied the five subroutines to a collection of problems which come from various applications. The first nine problems are associated with triangulations of two-dimensional domains and are typical of those arising in finite element applications. The reader is referred to [6] for a description of these nine problems. The remaining problems were

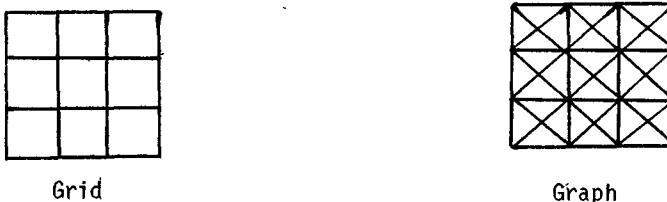


Fig. 3. A 4 by 4 grid and its associated graph

¹ Here and elsewhere, the notation $\lfloor y \rfloor$ means the largest integer less than or equal to y .

Table III. A Summary of the Information in Table II

(The first row is the *sum* of the corresponding columns of Table II. The last two rows are *averages* of the ratios computed from the appropriate columns of Table II. "Diameter" refers to the longest level found by any of the methods.)

	Original	Short circuiting	Strategy S1	Strategy S2	Strategy S3
Total time	10.09	7.00	9.02	5.52	5.42
Minimum length diameter	.984	.815	.984	.981	.884
Average length diameter	.996	.939	.995	.986	.977

kindly provided by Dr. Iain Duff of the AERE, Harwell, and include normal equations of sparse least squares problems arising in surveying and linear programming, and a set of random test matrices.

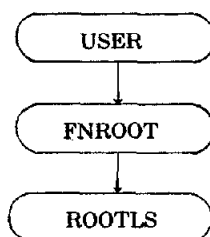
In general, the performance of any of the strategies on any of the problems is sensitive to the selection of the starting node. That is, any of them can be made to perform well by a judicious choice of starting node. Ideally, we should have run each implementation, on each graph, using *every* node in the graph as a starting node, and then averaged the results. However, this was prohibitively expensive, since many of our test problems have more than 1000 nodes. Instead, we selected a set of 10 starting nodes and averaged the results they generated.

The 10 nodes were obtained as follows: we first generated a level structure rooted at a pseudoperipheral node. The nodes of this level structure are returned in a vector LS , stored level by level in consecutive positions from 1 to $|X|$. The 10 starting nodes were then selected as $LS[(k \cdot |X|/10)]$, for $k = 1, 2, \dots, 10$. This would tend to choose nodes more or less sprinkled throughout the graph.

The summary of the information in Table II, which is contained in Table III, explains why we have chosen to provide an implementation of a pseudoperipheral node finder which uses strategy S2. Its worst case behavior and average behavior was very close to the best of the strategies, and it was considerably faster.

APPENDIX

We include here a Fortran implementation of a pseudoperipheral node finder which uses strategy S2. It consists of a pair of subroutines whose control relationship is as depicted below.



The user calls the subroutine **FNROOT**, which returns a pseudoperipheral node and its associated level structure. The subroutine **ROOTLS** is used repeatedly by **FNROOT**, and generates level structures from a given input node.

In the implementation, the graph is stored as a sequence of adjacency lists. The lists are kept consecutively in a single array, ADJNCY. An index array, XADJ, is used to mark the beginnings of the adjacency lists. Thus, the nodes adjacent to the i th node are given by $\{ADJNCY(k) \mid k = XADJ(i), XADJ(i) + 1, \dots, XADJ(i + 1) - 1\}$. The rooted level structure is stored in the same manner by an array pair (LS, XLS).

Sometimes computer programs which manipulate graphs store the graph in an N by μ rectangular array, with each adjacency list stored in a row. Here μ must be large enough to accommodate the largest adjacency list, which can result in a substantial waste of storage if the degrees of the nodes vary a lot. Thus, we adopted our scheme instead.

The subroutines FNROOT and ROOTLS are designed to operate on section subgraphs prescribed by the mask vector MASK, which is an input array. The subroutines only consider those nodes i for which $MASK(i) \neq 0$.

```

C*****
C*****
C***** FNROOT .... FIND PSEUDO-PERIPHERAL NODE *****
C*****
C*****
C
C  PURPOSE - FNROOT IMPLEMENTS A MODIFIED VERSION OF THE
C            SCHEME BY GIBBS, POOLE, AND STOCKMEYER TO FIND PSEUDO-
C            PERIPHERAL NODES.  IT DETERMINES SUCH A NODE FOR THE
C            MASKED COMPONENT CONTAINING A GIVEN NODE IN THE GRAPH.
C
C  INPUT PARAMETERS -
C      ADJNCY - THE ADJACENCY STRUCTURE VECTOR.  SIZE IS TWO
C               TIMES THE NUMBER OF EDGES.
C      XADJ   - THE INDEX VECTOR TO ADJNCY.  SIZE = N+1.
C               THE NBRS OF NODE I IS GIVEN BY ADJNCY(K), WHERE
C               K RUNS FROM XADJ(I) TO XADJ(I+1)-1.
C      MASK   - THE MASK VECTOR USED TO DEFINE A SECTION GRAPH.
C               FNROOT IGNORES THOSE NODES WITH MASK = 0.
C
C  UPDATED PARAMETER -
C      ROOT   - ON INPUT, IT DEFINES THE MASKED COMPONENT WHERE
C               A PSEUDO-PERIPHERAL NODE IS REQUIRED.  ON RETURN,
C               IT CONTAINS THE NODE THUS OBTAINED.
C
C  OUTPUT PARAMETERS -
C      NLVL   - IS THE NUMBER OF LEVELS IN THE LEVEL STRUCTURE
C               ROOTED AT THE NODE ROOT.
C      LS     - THE LEVEL STRUCTURE VECTOR.  SIZE = N.
C      XLS    - THE INDEX VECTOR TO LS.  SIZE = N+1.
C               NODES IN LEVEL I ARE GIVEN BY LS(K), FOR K EQUAL
C               TO XLS(I), XLS(I)+1, ..., XLS(I+1)-1.
C
C  PROGRAM SUBROUTINES -
C      ROOTLS.
C
C*****
C
C      SUBROUTINE FNROOT(ROOT, XADJ, ADJNCY, MASK,
C1                      NLVL, XLS, LS)
C
C*****
C
C      INTEGER XADJ(1), ADJNCY(1), MASK(1), XLS(1), LS(1),
C1          ROOT, NLVL, CCSIZE

```



```

C
C*****
C
      SUBROUTINE ROOTLS(ROOT, XADJ, ADJNCY, MASK, NLVL, XLS, LS)
C*****
C
      INTEGER XADJ(1), ADJNCY(1), MASK(1), XLS(1), LS(1),
1         ROOT, NLVL, LBEGIN, LVLEND, LVSIZE,
2         NODE, LNBR, JSTRT, JSTOP
C*****
C
C      INITIALIZATION ....
C      -----
C      MASK(ROOT) = 0
C      LS(1) = ROOT
C      NLVL = 0
C      LVLEND = 0
C      LNBR = 1
C
C      -----
C      LBEGIN IS THE POINTER TO THE BEGINNING OF PRESENT
C      LEVEL, AND LVLEND POINTS TO THE END OF THIS LEVEL.
C      -----
200  LBEGIN = LVLEND + 1
      LVLEND = LNBR
      NLVL = NLVL + 1
      XLS(NLVL) = LBEGIN
C      -----
C      GENERATE THE NEXT LEVEL BY FINDING ALL THE MASKED
C      NEIGHBORS OF NODES IN PRESENT LEVEL.
C      -----
      DO 400 I = LBEGIN, LVLEND
        NODE = LS(I)
        JSTRT = XADJ(NODE)
        JSTOP = XADJ(NODE + 1) - 1
        IF ( JSTOP .LT. JSTRT ) GO TO 400
C
        DO 300 J = JSTRT, JSTOP
          NBR = ADJNCY(J)
          IF (MASK(NBR) .EQ. 0) GO TO 300
          LNBR = LNBR + 1
          LS(LNBR) = NBR
          MASK(NBR) = 0
300    CONTINUE
400  CONTINUE
C      -----
C      COMPUTE THE CURRENT LEVEL WIDTH.
C      IF IT IS NONZERO, GENERATE NEXT LEVEL.
C      -----
      LVSIZE = LNBR - LVLEND
      IF (LVSIZE .GT. 0) GO TO 200
C
      XLS(NLVL+1) = LVLEND + 1
C      -----
C      RESET MASK TO ONE FOR THE NODES IN THE LEVEL STRUCTURE.
C      -----
      DO 500 I = 1, LNBR
        NODE = LS(I)
        MASK(NODE) = 1
500  CONTINUE
C
      RETURN
END

```

ACKNOWLEDGMENTS

The authors are especially grateful to Prof. N.E. Gibbs, of the College of William and Mary, for suggesting strategies S2 and S3 as alternatives to strategy S1.

REFERENCES

1. ARANY, I, SMYTH, W.F., AND SZODA, L. An improved method for reducing the bandwidth of sparse symmetric matrices. In Information Processing 71, North-Holland Pub. Co., Amsterdam, 1972.
2. BERGE, C. *The Theory of Graphs and its Applications*. Wiley, New York, 1962.
3. CRANE, H.L., JR., GIBBS, N.E., POOLE, W.G., JR., AND STOCKMEYER, P.K. Algorithm 508. Matrix bandwidth and profile reduction. *ACM Trans. Math. Software* 2, 4 (Dec. 1976), 375-377.
4. CUTHILL, E., AND MCKEE, J. Reducing the bandwidth of sparse symmetric matrices. Proc. 24th Nat. Conf., ACM Publ. P-69, Assoc. Comput. Mach., New York, 1969.
5. EVERSTINE, G.C. The BANDIT computer program for the reduction of matrix bandwidth for NASTRAN. NSRDC Rep. 3827, 1972.
6. GEORGE, A., AND LIU, J.W.H. Algorithms for matrix partitioning and the numerical solution of finite element systems. *SIAM J. Numer. Anal.* 15 (1978), 297-327.
7. GEORGE, A., AND LIU, J.W.H. An automatic nested dissection algorithm for irregular finite element problems. *SIAM J. Numer. Anal.* 15 (1978), 1053-1069.
8. GIBBS, N.E., POOLE, W.G., AND STOCKMEYER, P.K. An algorithm for reducing the bandwidth and profile of a sparse matrix. *SIAM J. Numer. Anal.* 13 (1976), 236-250.
9. LIU, J.W.H. On reducing the profile of sparse symmetric matrices. Rep. CS-76-07, Dept. of Comput. Sci., U. of Waterloo, Waterloo, Ont., Canada, 1976.
10. LIU, J.W.H., AND SHERMAN, A.H. Comparative analysis of the Cuthill-McKee and the reverse Cuthill-McKee ordering algorithms for sparse matrices. *SIAM J. Numer. Anal.* 12 (1975), 198-213.
11. SMYTH, W.F., AND ARANY, I. Another algorithm for reducing bandwidth and profile of a sparse matrix. Proc. AFIPS 1976 NCC, AFIPS Press, Montvale, N.J., pp. 987-994.
12. SMYTH, W.F., AND BENZI, W.M.L. An algorithm for finding the diameter of a graph. In IFIP Congress 74, Vol. 3 North-Holland Pub. Co., Amsterdam, 1974, pp. 500-503.

Received November 1976, revised November 1978